

HOL 4: Linux File Permissions

Author: Dr. Sandeep Kumar Sharma

Learning Objectives

The objective of this Hands-On Lab (HOL) is to help **beginner-level Linux users** clearly understand:

- What file and directory permissions are in Linux
 - Why permissions are required in an operating system
 - Different types of permissions available in Linux
 - How to view permissions using commands
 - How to assign permissions to files and directories
-

Learning Outcomes

After completing this lab, learners will be able to:

- Explain Linux permissions in simple words
 - Identify read, write, and execute permissions
 - Understand owner, group, and others
 - Read and understand `ls -l` output confidently
 - Change permissions on files and directories safely
 - Apply correct permissions based on real-life use cases
-

Step 1: What Are Permissions in Linux?

In Linux, **permissions** decide **who can do what** with a file or a directory.

Whenever a file or directory is created, Linux automatically assigns permissions to it. These permissions control:

- Who can open the file
- Who can modify the file
- Who can run the file as a program

Think of permissions like **locks on a door**. Not everyone should be allowed to open, modify, or use everything on the system.

Step 2: Why Do We Need Permissions?

Permissions are required to keep the system **secure and organized**.

On a Linux server:

- Many users work at the same time
- Some files are private
- Some files are shared
- Some files are critical for the system

Permissions make sure that:

- Normal users cannot damage system files
 - Users cannot see each other's private data
 - Only authorized users can run or modify programs
-

Step 3: Types of Permissions in Linux

Linux has **three basic permissions**.

Read (r) means the permission to open and view the content of a file. For a directory, read permission allows listing the files inside it.

Write (w) means the permission to modify a file. For a directory, write permission allows creating, deleting, or renaming files inside it.

Execute (x) means the permission to run a file as a program. For a directory, execute permission allows entering the directory using the `cd` command.

These permissions behave slightly differently for files and directories, which is why understanding them is very important.

Step 4: Permission Categories – Who Gets the Permission?

Linux assigns permissions to **three categories**.

The **owner** is the user who created the file. The owner usually has the highest level of access.

The **group** represents a set of users who share access to the file.

Others means everyone else on the system who is not the owner and not part of the group.

Linux checks permissions in this order: **owner** → **group** → **others**.

Step 5: Checking File and Directory Permissions

To check permissions, Linux provides the `ls -l` command.

Command

```
ls -l
```

Example output:

```
-rwxr-xr-- 1 sandeep devgroup 1234 file.sh
```

Step 6: Understanding `ls -l` Output (Very Important)

Let us break the permission part:

```
-rwxr-xr--
```

- The first character shows the type (`-` for file, `d` for directory)
 - The next three characters (`rwx`) are permissions for the **owner**
 - The next three (`r-x`) are permissions for the **group**
 - The last three (`r--`) are permissions for **others**
-

Step 7: Meaning of Each Permission with Use Case

If a file has **read permission**, the user can open and read it, like reading a text file or configuration file.

If a file has **write permission**, the user can modify or delete the file, which is required when editing files.

If a file has **execute permission**, the user can run the file as a program or script.

For directories:

- Read → list files
 - Write → create or delete files
 - Execute → enter the directory
-

Step 8: Changing Permissions Using `chmod` (Beginner Way)

The `chmod` command is used to change permissions.

Example: Give execute permission to owner

```
chmod u+x script.sh
```

This command allows the file owner to execute the script.

Step 9: Numeric Permission Concept (Simple Explanation)

Linux also supports numeric permissions:

- Read = 4
- Write = 2
- Execute = 1

Common examples:

- `755` → owner full access, others read & execute
- `644` → owner read/write, others read only

Command Example

```
chmod 644 file.txt
```

Step 10: Giving Permissions to a Directory

Example

```
chmod 755 mydir
```

Explanation:

- Owner can enter and modify
 - Others can enter and view
-

Step 11: Checking Permissions for a Specific File

```
ls -l file.txt
```

Always verify permissions after changing them.

Step 12: Real-Life Beginner Scenario

Scenario: A developer needs to run a script but cannot execute it.

Solution:

```
chmod +x deploy.sh
```

Now the script can be executed.

End of HOL 4

This lab builds a **strong beginner-level understanding** of Linux permissions and prepares learners for user management, security, and DevOps topics.